Documentación técnica

WebAppLLM

Proyecto de Sistemas Distribuidos

20 de octubre de 2026

Resumen

Este documento describe la estructura y los flujos principales de WebAppLLM, una aplicación web Java basada en Spring Boot. La aplicación combina páginas Thymeleaf, persistencia JPA, autenticación Spring Security, mensajería STOMP sobre WebSocket y una interfaz conversacional que integra Google Gemini mediante Spring AI. La descripción se ha elaborado a partir del código fuente y la configuración presentes en el repositorio.

Índice

1. Resumen del sistema **2**
1.1. Tecnologías observadas 2

2. Organización del código **2**

3. Flujos funcionales **2**
3.1. Inicio de sesión y gestión de usuarios 2
3.2. Chat con modelo de lenguaje 3
3.3. Mensajería WebSocket 3

4. Configuración y ejecución **3**

5. Vistas principales **3**

6. Límites y aspectos que conviene revisar **3**

7. Mapa de archivos de referencia **4**

8. Compilar este documento **4**

1 Resumen del sistema

WebAppLLM es una aplicación web monolítica organizada por capas. El proceso arranca en `com.sistemasdistr.basico.Webappllm`; Spring Boot detecta los componentes y configura el contexto de la aplicación. Las páginas del servidor se renderizan con Thymeleaf. Las cuentas se almacenan mediante repositorios Spring Data JPA, y el módulo de chat delega la generación de respuestas a Spring AI y al modelo Google Gemini.

1.1. Tecnologías observadas

- Java 25 y Spring Boot 4.0.3 (declarados en `pom.xml`).
- Spring MVC y Thymeleaf para páginas y formularios.
- Spring Data JPA con conector MySQL.
- Spring Security con formulario de acceso y codificación BCrypt.
- Spring WebSocket con STOMP y SockJS para mensajería en tiempo real.
- Spring AI 1.1.2 y Google GenAI para chat con herramientas.
- Maven Wrapper para compilar y ejecutar el proyecto sin instalar Maven globalmente.

2 Organización del código

Paquete	Responsabilidad principal
`controller`	Controladores MVC para usuarios, autenticación, páginas y WebSocket.
`controller.llm`	Endpoints y vistas de la conversación con el asistente.
`service`	Casos de uso de usuarios, claves y autenticación.
`service.llm`	Integración con Gemini y consultas FAQ.
`model` / `dto`	Entidades persistentes y objetos de transferencia.
`repository`	Acceso a datos JPA y repositorios de FAQ.
`faq` / `datallm`	Herramientas e intenciones de dominio utilizadas por los asistentes.
`config`	Seguridad, memoria conversacional, gestión de excepciones y WebSocket.
`util`	Respuestas, uso de tokens, identificación de idioma y mensajes alternativos.
`src/main/resources/templates`	Vistas Thymeleaf para páginas, usuarios y chat.

3 Flujos funcionales

3.1. Inicio de sesión y gestión de usuarios

`SecurityConfig` define un inicio de sesión por formulario en `/login`; permite el acceso a la portada y al registro, y exige autenticación para las demás rutas. `CustomUserDetailsService` obtiene la cuenta por nombre de usuario mediante `UserRepository`. El proveedor de autenticación compara la contraseña con BCrypt.

`AppUserController` expone operaciones de listado, alta, edición y eliminación en la ruta `/users`. `UserService` aplica reglas de alta y actualización, codifica contraseñas y delega persistencia al repositorio. `KeyService` genera pares RSA de 2048 bits y codifica ambas claves en Base64; el servicio las asigna al perfil de usuario.

3.2. Chat con modelo de lenguaje

`AppLlmController` presenta las vistas del chat y recibe las preguntas. `GeminiService` crea la petición con una instrucción de sistema, la pregunta, el identificador de conversación y las herramientas `HotelFaqTool` y `RoomPriceTool`. `AiConfig` comparte una memoria de ventana de mensajes configurada para el cliente de chat.

Las herramientas consultan repositorios de FAQ de hotel y precios. La respuesta se representa como `AiChatResponse`, que contiene el texto y, cuando está disponible, el recuento de tokens de entrada, salida y total. El servicio detecta errores de cuota y fallos temporales para devolver mensajes alternativos; el manejador global traduce la excepción de cuota a HTTP 429.

3.3. Mensajería WebSocket

`WebSocketConfig` habilita STOMP, configura los prefijos de aplicación `/app`, de suscripción `/topic` y `/queue`, y el prefijo de destinos de usuario `/user`. `AppWebSocketController` procesa mensajes y registros recibidos en los destinos de aplicación y construye objetos `OutputMessage`. Los endpoints incluyen `/chat`, `/register`, `/chathash` y `/queuechat`, con soporte SockJS.

4 Configuración y ejecución

La configuración de base de datos y puerto se externaliza mediante variables de entorno referenciadas en `src/main/resources/application.properties`. Antes de arrancar, deben definirse al menos el nombre de aplicación, la URL y credenciales de la base de datos, el driver, el modo DDL de JPA y el puerto. La integración de Google GenAI también necesita una clave de API válida, configurada fuera del repositorio.

```
# Windows (PowerShell)
.\mvnw.cmd spring-boot:run

# Linux/macOS
./mvnw spring-boot:run
```

Para empaquetar un JAR ejecutable:

```
.\mvnw.cmd clean package
```

El proyecto incluye un `Dockerfile`; la imagen puede construirse desde la raíz del repositorio con Docker instalado.

5 Vistas principales

Las plantillas se encuentran bajo `src/main/resources/templates`. Las páginas de autenticación y usuarios están en `users`; el chat con IA está en `llm`; la portada y las páginas de mensajería usan las plantillas situadas en la raíz. Hay también plantillas de producto. La presencia de una plantilla no implica por sí sola que el flujo esté conectado o completo: para verificar cada pantalla deben seguirse sus rutas en el controlador correspondiente.

6 Límites y aspectos que conviene revisar

- Revisar las credenciales y claves de servicio en propiedades: los secretos deben llegar mediante variables de entorno o un gestor de secretos, nunca quedar versionados en texto claro.
- Verificar las rutas WebSocket y las políticas de origen antes de desplegar; algunas configuraciones aceptan cualquier origen.

- Evaluar el uso de una memoria de chat compartida como singleton: la separación de conversaciones depende de que cada petición suministre correctamente su identificador de conversación.
- Confirmar que los métodos de autenticación no impriman contraseñas ni hashes en la salida estándar o los registros.
- Las propiedades de JPA y las variables de entorno deben ajustarse al entorno objetivo. La aplicación depende de una instancia compatible con MySQL y del acceso a Google GenAI.

7 Mapa de archivos de referencia

- Arranque: `src/main/java/com/sistemasdistr/basico/Webappllm.java`.
- Dependencias y versiones: `pom.xml`.
- Propiedades: `src/main/resources/application.properties`.
- Seguridad: `config/SecurityConfig.java` y `config/CustomUserDetailsService.java`.
- Chat: `controller/llm/AppLlmController.java`, `service/llm/GeminiService.java` y `config/llm/AiConfig.java`.
- WebSocket: `config/websocket/WebSocketConfig.java` y `controller/AppWebSocketController.java`.
- Vistas: `src/main/resources/templates/`.

8 Compilar este documento

Este proyecto utiliza LaTeX estándar y puede compilarse con pdfLaTeX:

```
pdflatex main.tex
pdflatex main.tex
```

La segunda ejecución actualiza el índice. Los archivos fuente de la aplicación se consultan en el directorio padre del proyecto de documentación.